

Hackathon: Analysis of APSA eJobs Listings

Stern Team · EPPS 6323, Knowledge Mining · 4/2/2026

Text Mining Pipeline

The goal of our pipeline was to transform 113 PDF files into a structured text corpus that supports analysis at the level of individual job postings. Because each PDF contains multiple postings, we segmented extracted text into job-level units. Each unit was then enriched with structured metadata—including institution, rank, subfield, and posting date—enabling both text analysis and comparisons across time, category, and institution.

PDF Extraction

Text was extracted from PDFs using functions from the `pdftools` package, following methodologies adapted from course sample scripts. Error handling was incorporated to ensure that unreadable or malformed PDFs did not interrupt the pipeline.

Ad Segmentation

Initial segmentation attempts used `quanteda::corpus_segment()`. However, job postings were delimited by an eJobs ID appearing at the end of each listing rather than the beginning, making this function unsuitable.

Instead, regular expressions were applied prior to corpus construction to split documents into individual job postings. This preprocessing step ensured that each posting could be treated as a distinct document, enabling accurate assignment of document-level variables (`docvars`) and supporting downstream analyses such as subsetting, keyness comparisons, and temporal trend analysis.

Metadata Extraction

Each listing contained structured footer fields like posting date, application deadline, salary, and eJobs ID. These were extracted via regular expression from each ad with a high rate of success. Header fields of subfield and rank were pulled the same way. The institution name, however, was not a structured field.

Metadata Cleaning

The institution name was not consistently formatted and required additional processing. An initial positional approach—extracting the line preceding the “Rank:” label—resulted in high missingness (~91%).

In response, a manual review process was undertaken using Claude Code to inspect the extracted ads with missing institution values. Alternative strategies like extracting the institution name from the email address in the application instructions (e.g., `hr@georgetown.edu`) and parsing it from the equal opportunity employer statement

typically appearing in the ad footer (e.g., "*X University is an equal opportunity employer*") filled gaps, allowing for consistent labelling in the final corpus.

Corpus Construction and Tokenization

Individual ads were loaded into a `quanteda` corpus object, with all extracted metadata attached as document-level variables (`docvars`).

The corpus was then tokenized using `quanteda`'s `tokens()` function, with punctuation, numbers, symbols, and URLs removed. Tokens were lowercased and English stopwords were removed using `quanteda`'s built-in stopwords list. An additional custom stopwords list was applied to remove terms deemed analytically uninformative for this corpus (Appendix A.1.).

The cleaned token list was converted to a document-feature matrix (DFM) using `dfm()`, with features appearing fewer than five times across the corpus or in fewer than two documents removed via `dfm_trim()`.

AI-Assisted Code Development Disclosure

Claude (Anthropic, `claude-sonnet-4-6`) was used as a coding assistant during the development of the data processing and analysis pipeline described in this section. AI assistance was used to draft and debug R code for PDF extraction, corpus segmentation, metadata extraction, and `quanteda` preprocessing. All generated code was reviewed, tested, and modified by the authors prior to use. Analytical decisions — including segmentation strategy, stopwords list construction, and variable definitions — were made by the research team. A sample of prompts submitted during this process are documented in Appendix A.7.

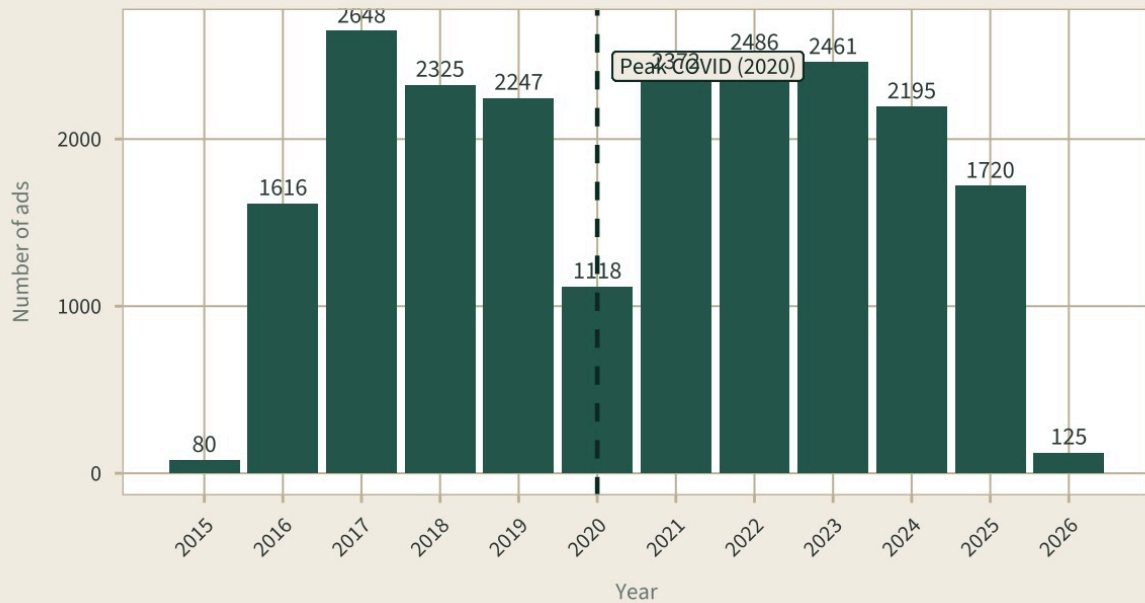
Analysis

Corpus Overview

The summary graphs describe the corpus as a whole over a period of 10 years: ads by year and section distribution by volume and year. The impact of the pandemic during 2020 is apparent in all of the graphs, with the overall ads and most sections increasing postings in 2021 and 2022 once sectors recovered from the impact of the pandemic. Administration postings have the largest volume, and comparative politics, international relations, methodology, and other also have large volumes during specific years.

Number of Political Science Job Ads Posted Per Year

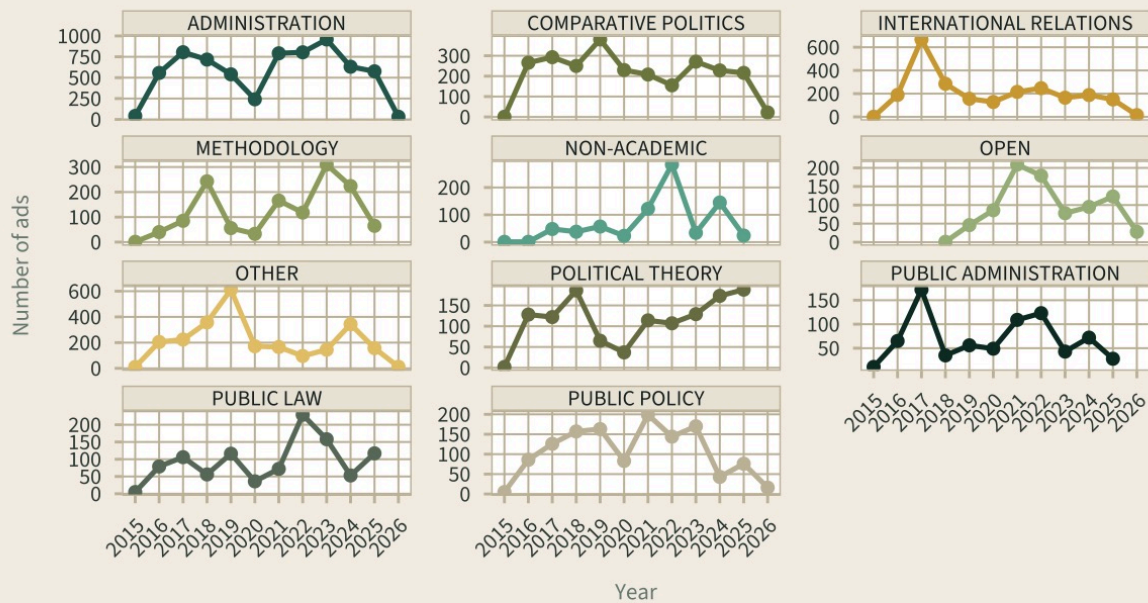
Total listings across all sections



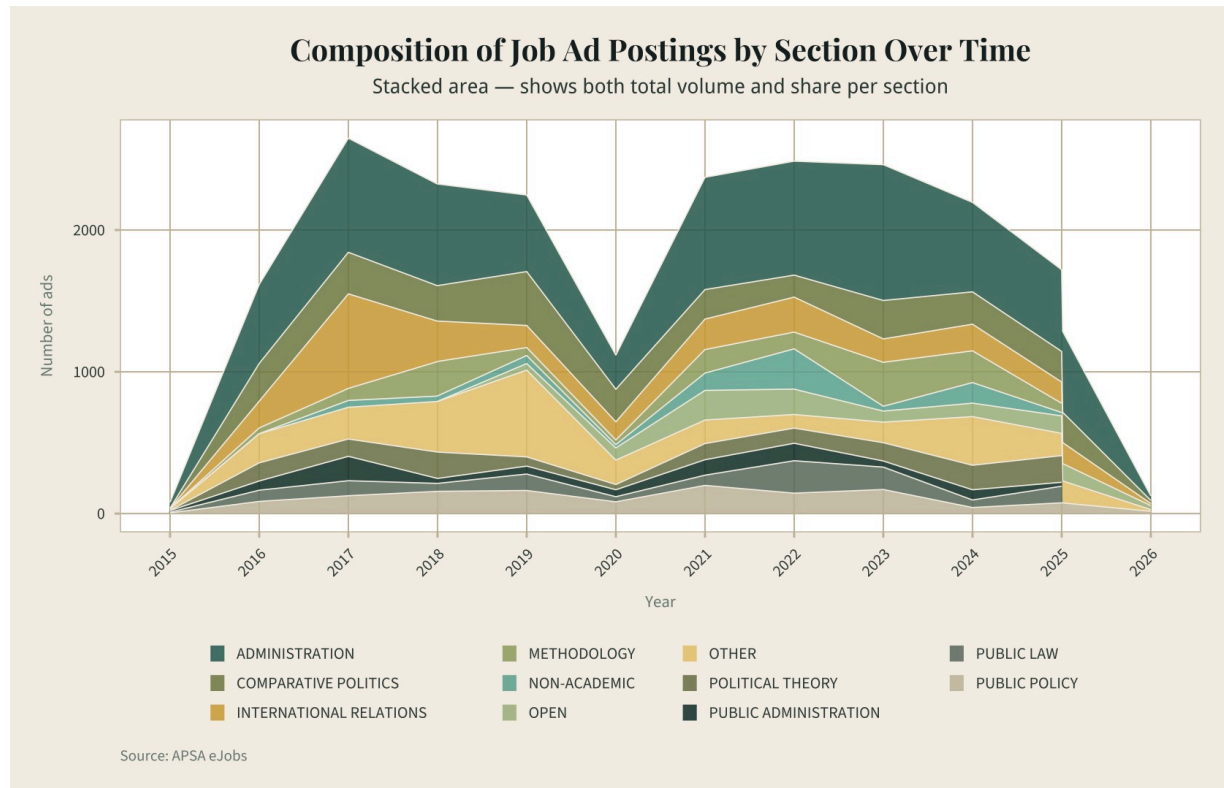
Source: APSA eJobs

Job ad Postings by Section Over Time

Y-axis is free-scaled per section to highlight relative trends



Source: APSA eJobs

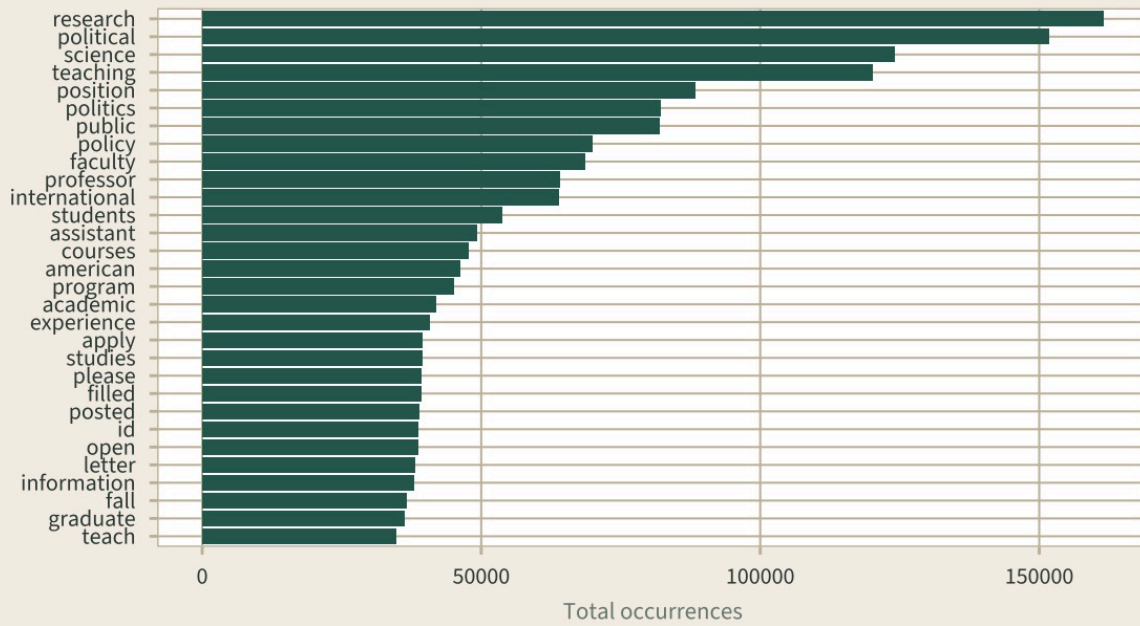


Most Frequent Terms

Analyzing the most common terms in the corpus overall shows that “research,” “political,” “science,” and “teaching” are by far the most frequently occurring terms. Further analysis of terms by section shows that “research” still predominates in every section as the top or almost top term, though several sections have a word from the heading that also predominates (e.g., “policy” in public policy and “public” in public administration). Overall, these graphs reveal a shared vocabulary and commonality amongst the sections of political science in job advertising.

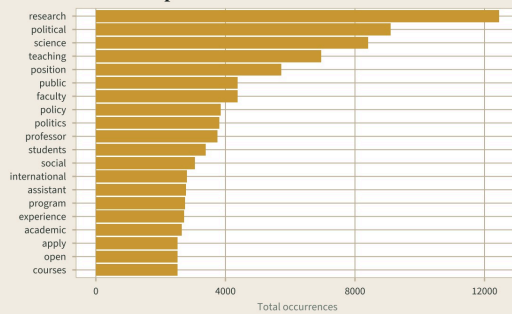
Most common words in political science job ads

All sections, all years



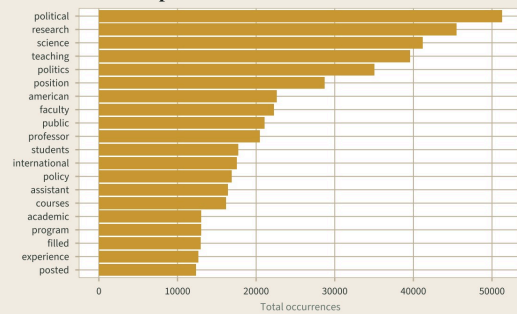
Source: APSA eJobs

Top words in METHODOLOGY ads



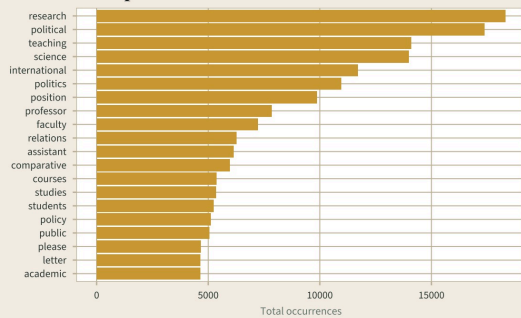
Source: APSA eJobs

Top words in ADMINISTRATION ads



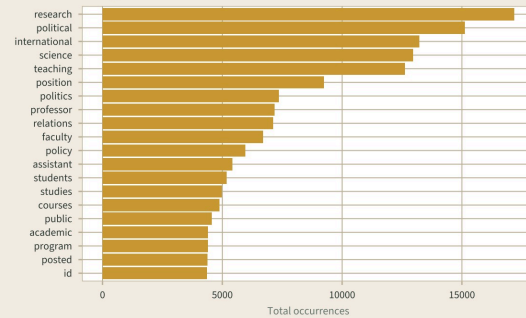
Source: APSA eJobs

Top words in COMPARATIVE POLITICS ads

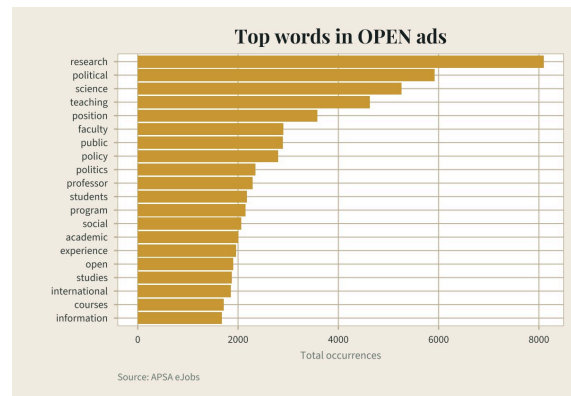
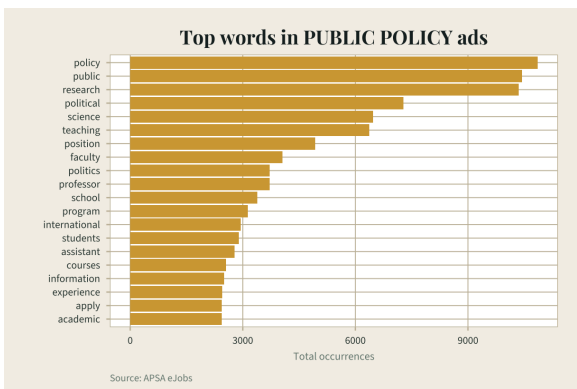
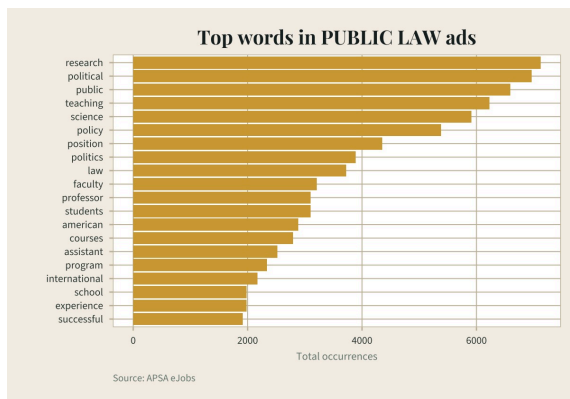
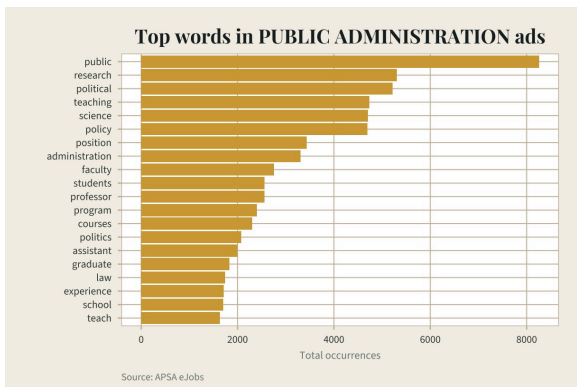
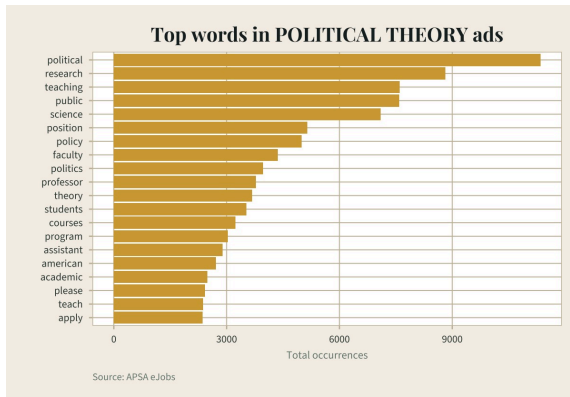
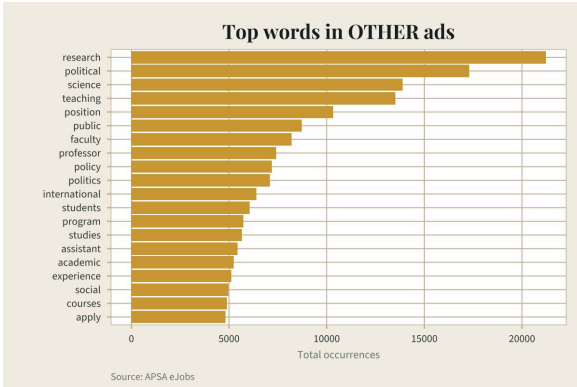
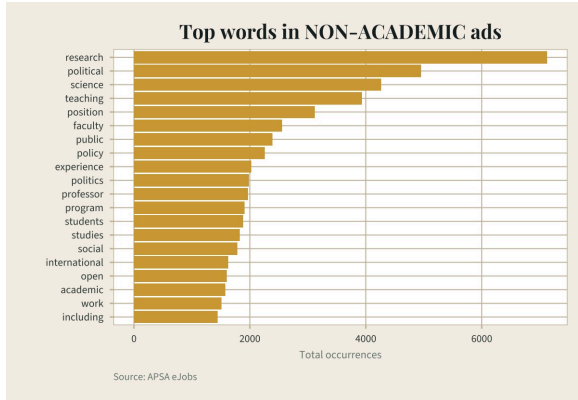


Source: APSA eJobs

Top words in INTERNATIONAL RELATIONS ads

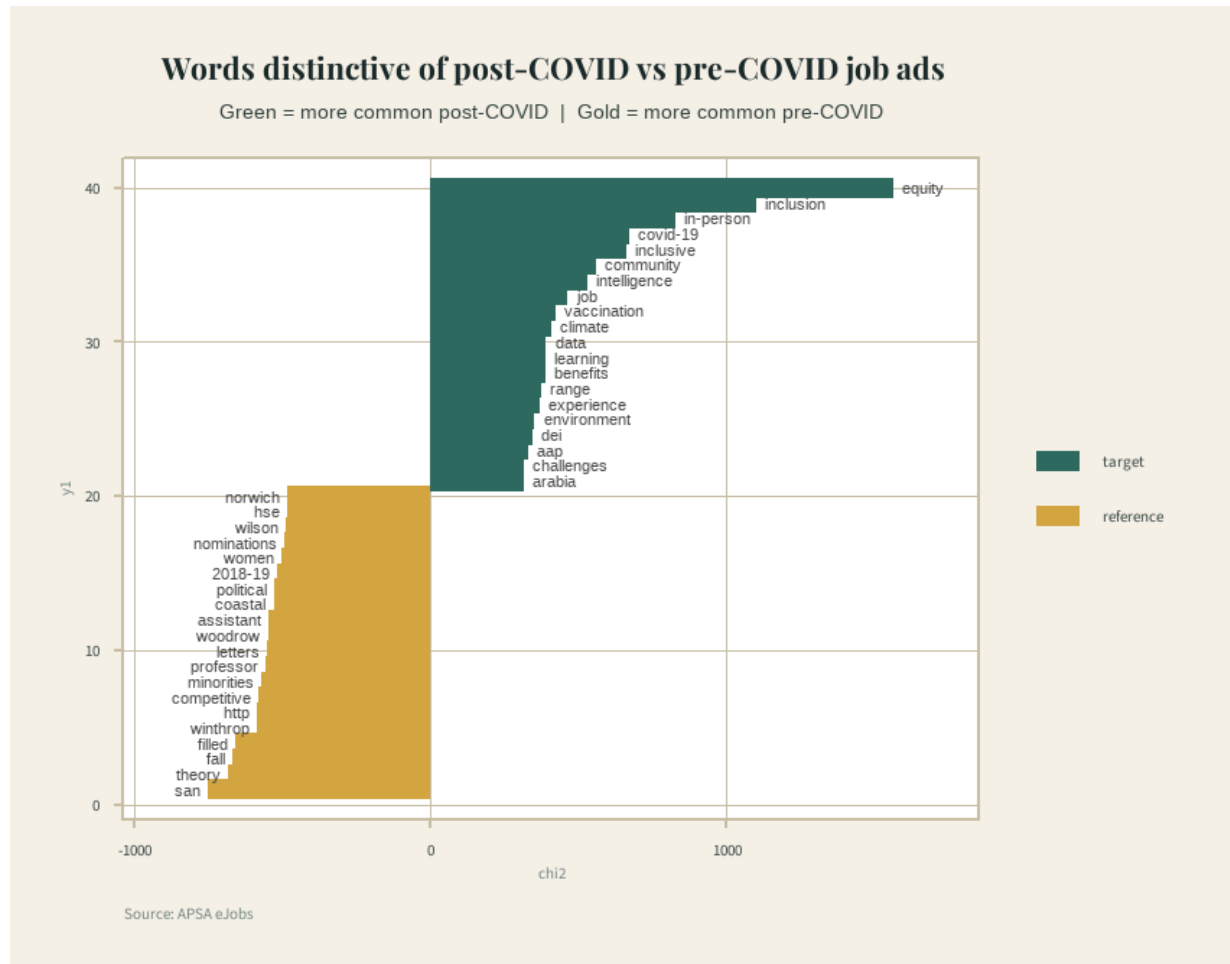


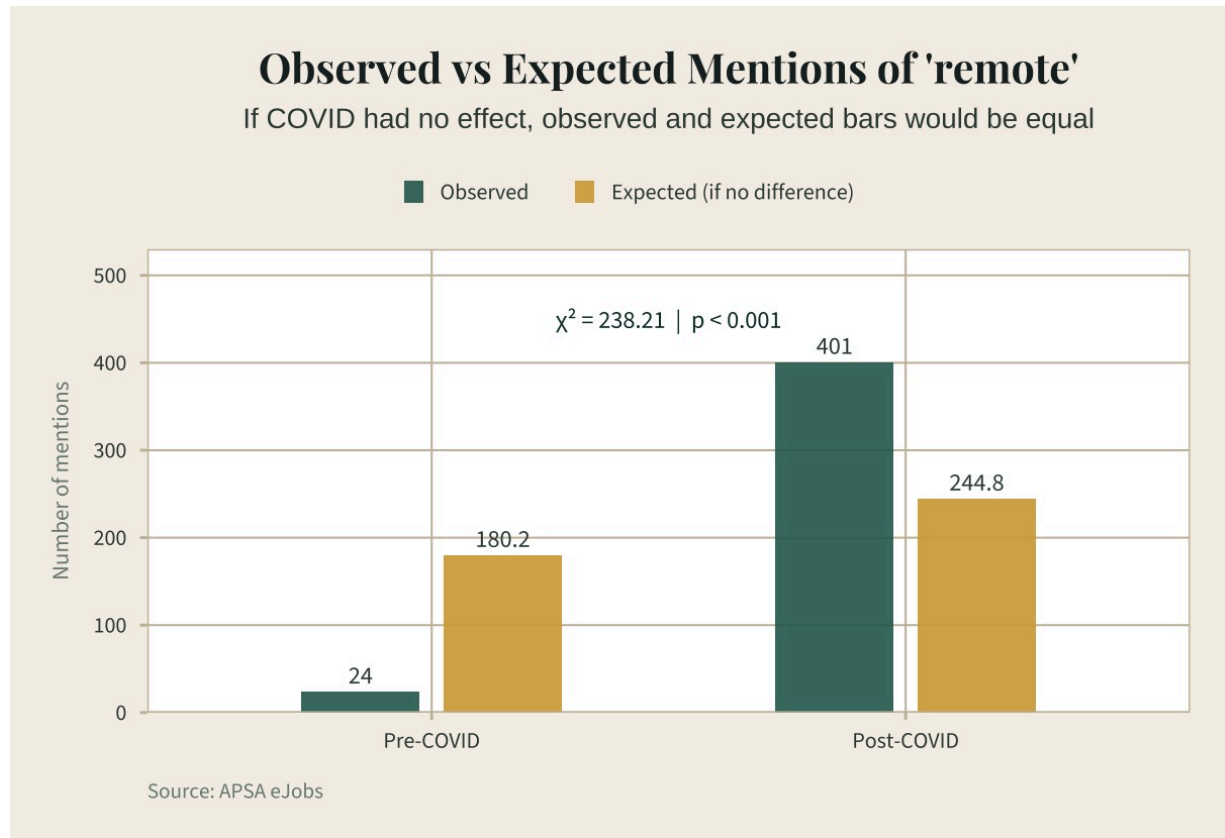
Source: APSA eJobs



Pre- vs. Post-COVID Keyness

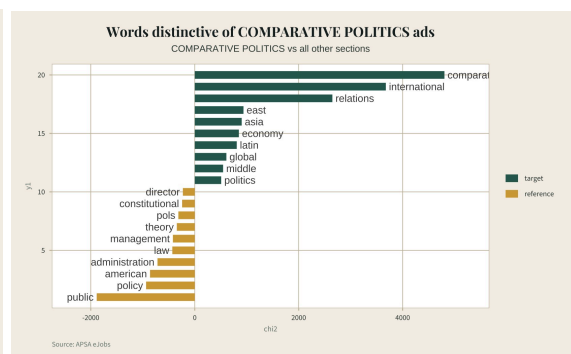
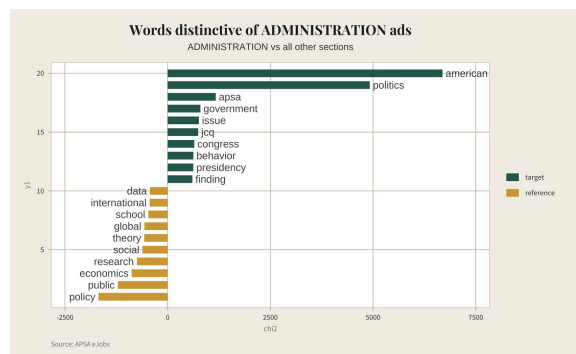
A keyness analysis comparing the distinctive vocabulary of pre-COVID (before March 2020) and post-COVID (March 2020 onward) job ads revealed interesting changes in the types of terms used following the pandemic. Although pre-COVID ads featured “women” and “minorities,” these terms seemed more clinical and job-focused. The post-COVID terms focused more on community and broader terms to describe inclusive hiring practices and environments. Additionally, the pandemic dramatically affected the advertisement of “remote” work opportunities.

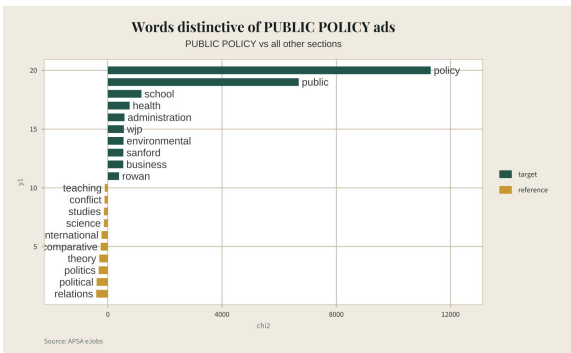
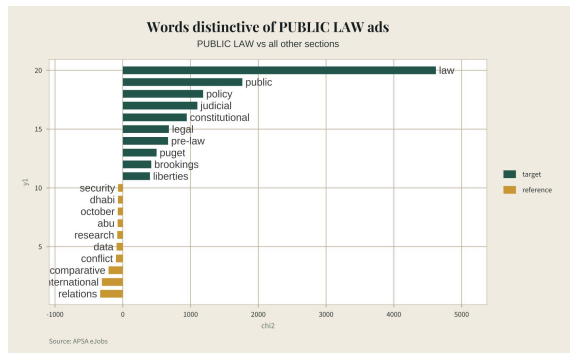
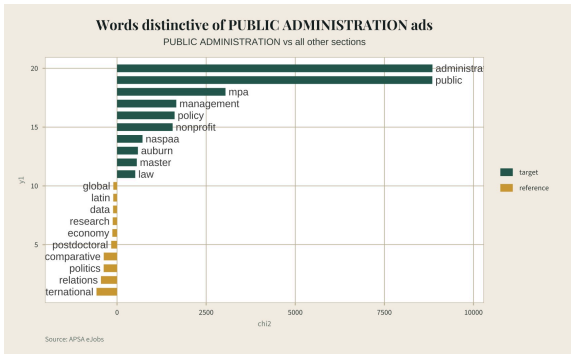
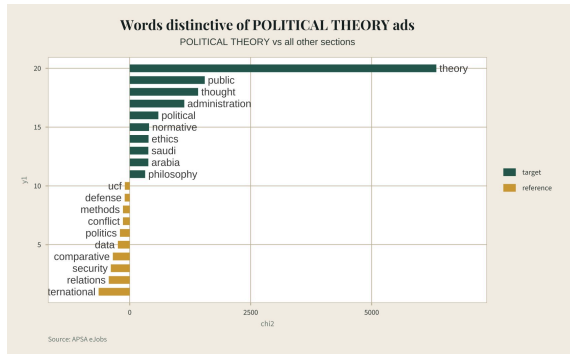
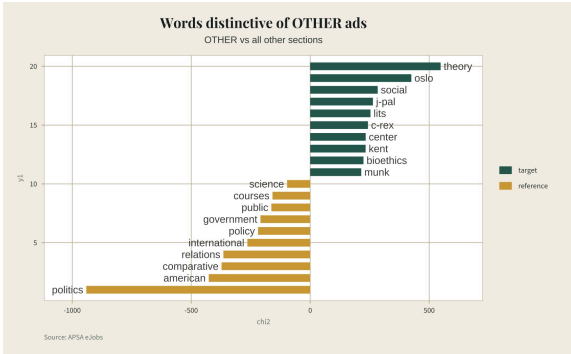
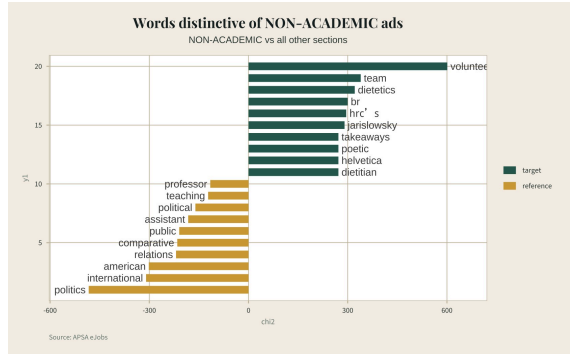
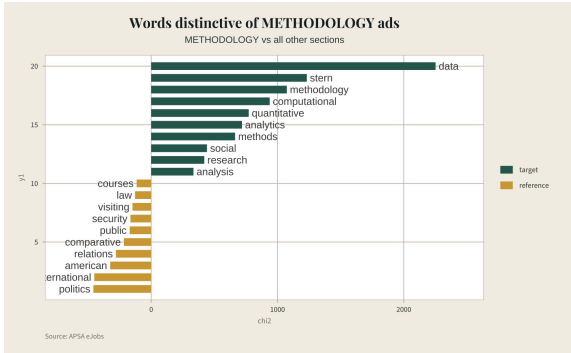
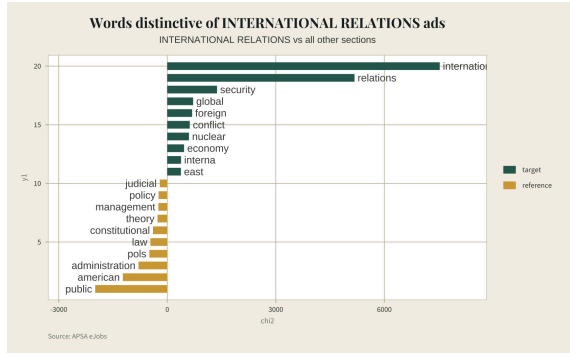


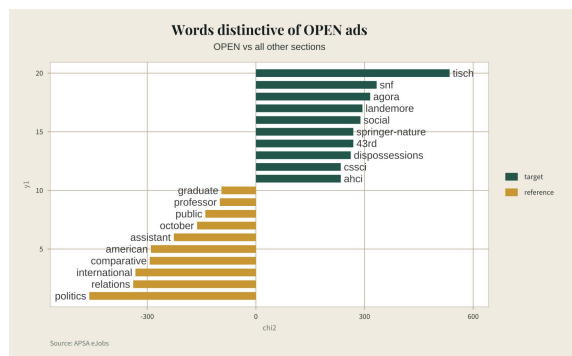


Subfield Distinctiveness

A keyness analysis of each section against all other sections shows which 10 terms are more specific to those fields of political science. Though the most common terms listed previously show that many sections share the same vocabulary, this analysis highlights the lexicon that clearly differentiates each subfield's job market. Some words in the sections are obvious, such as “comparative” in comparative politics or “data” in methodology, whereas others are more surprising, such as “bioethics” in other and “behavior” in administration.



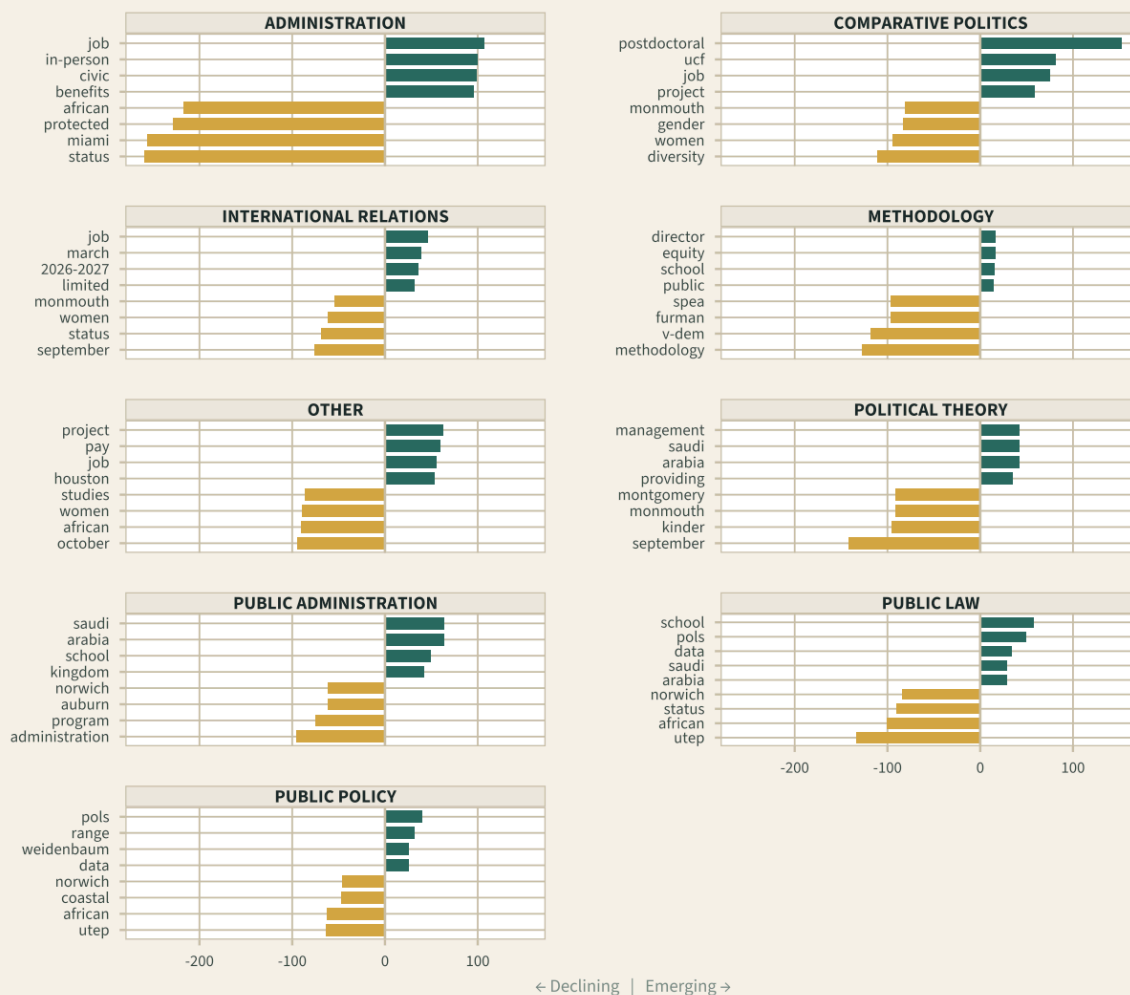




Emerging and Declining Keywords by Section

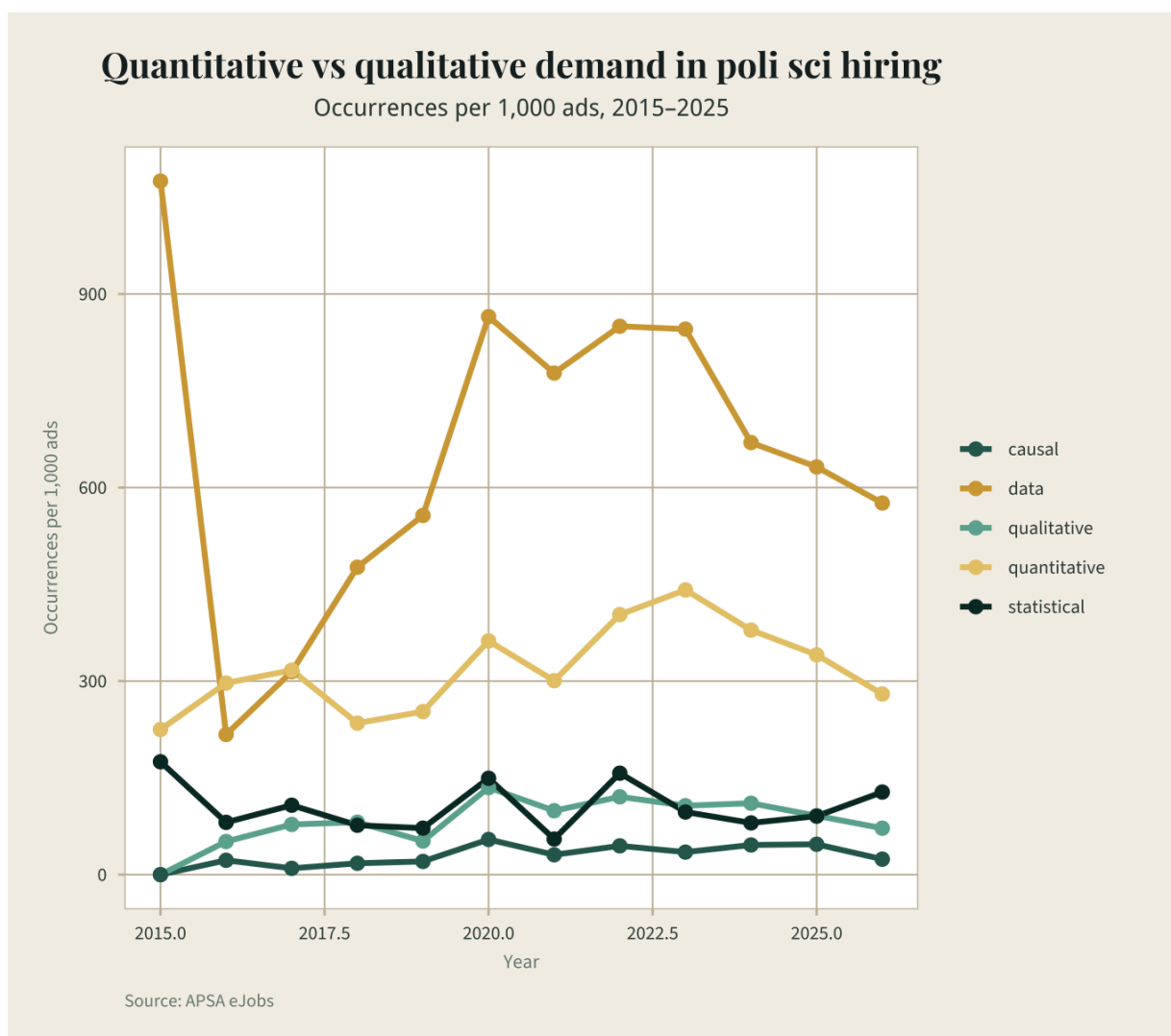
Earliest 2 vs latest 2 years within each section | χ^2 keyness, target = recent

Emerging Declining



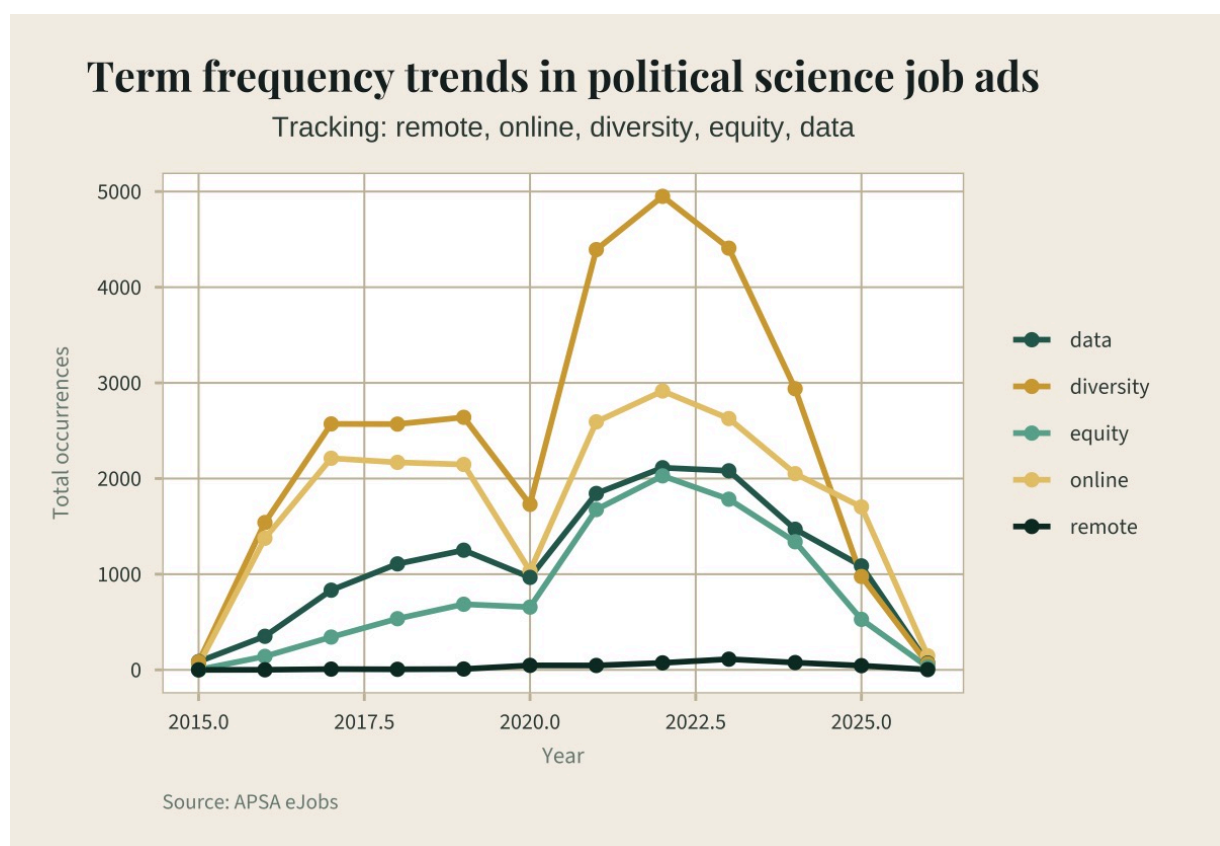
Qualitative vs Quantitative skill demand across time

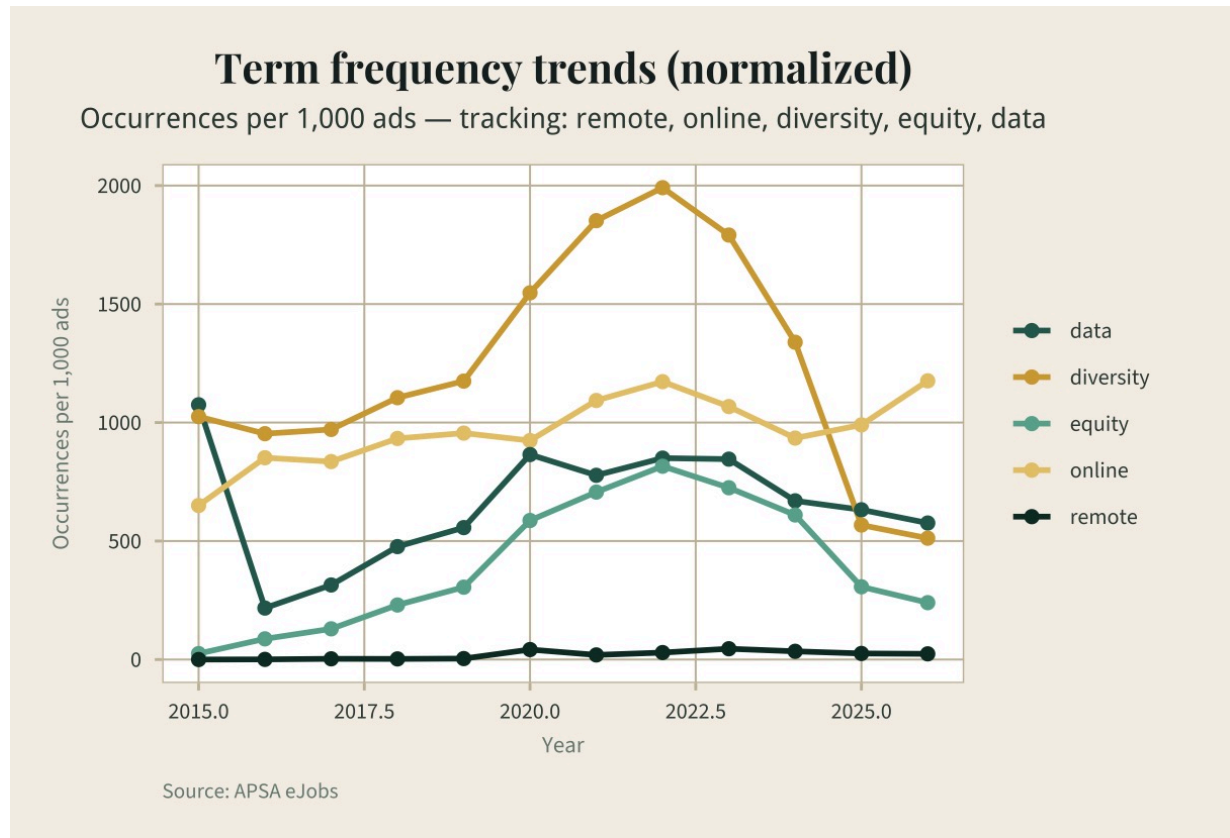
Frequency analysis of the terms “data”, “quantitative”, “qualitative”, “causal” and “statistical” from APSA job posting from 2015-2025 reveals that demand for data-related skills has grown over the past ten years. The balance between quantitative and qualitative research skills has remained broadly stable, although quantitative researchers are sought at twice or three times the rate as qualitative researchers. Terms like “causal” and “statistical” remain consistently low, suggesting that hiring teams utilize broader descriptions of methodological skills that they seek. It should be noted that years 2015 and 2026 contain far fewer ads than other years, so findings from those years should be interpreted cautiously. Together these trends suggest that data and quantitative skill demand remains high across the political sciences. With the rise of AI tools that are able to make these skills more accessible to a wider pool of applicants, it will be interesting to see how these trends change in years to come.



Longitudinal Term Trends

Tracking selected term frequencies per year (raw and normalized per 1,000 ads) shows the rise or decline of those terms across the time period. The data reveal that “diversity” was the most frequently occurring term generally until 2025, which likely correlates with the new presidential administration taking office and its focus on dismantling and defunding diversity, equity, and inclusion programs. Unrelatedly, “online” has had a strong presence and will likely continue to increase in frequency of occurrence. Though “remote” showed some importance in the wake of the pandemic, it does not figure prominently in comparison to other selected terms.



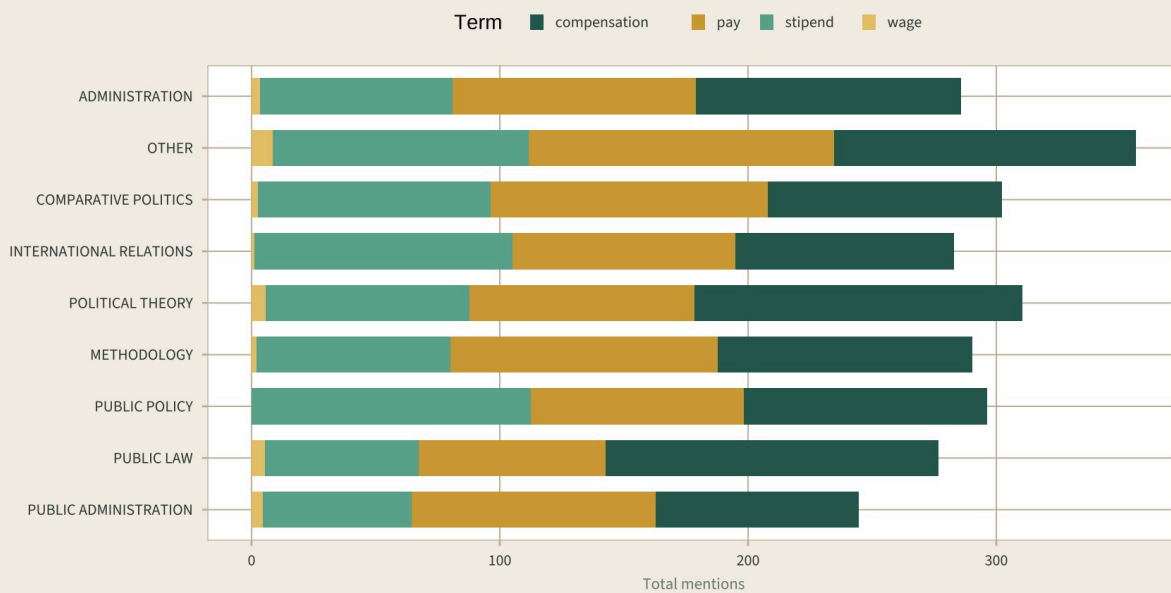


Salary and Wages

An analysis of terms surrounding salary and pay indicates that “compensation” is the most predominant term with “pay” also featuring prominently and “stipend” to a lesser extent. The impact of the pandemic can also be seen in the sheer number of times these terms increase in presence post-COVID, which is likely due to job postings increasing overall once sectors began to recover from the impact. A geographical analysis of the amount of salary offered by state shows that states vary widely in the median salary offered, which may correlate with the quality and cost of living in those states also. Surprisingly though, Montana offers the highest median salary of all states.

Salary and Pay-Related Term Mentions by Section

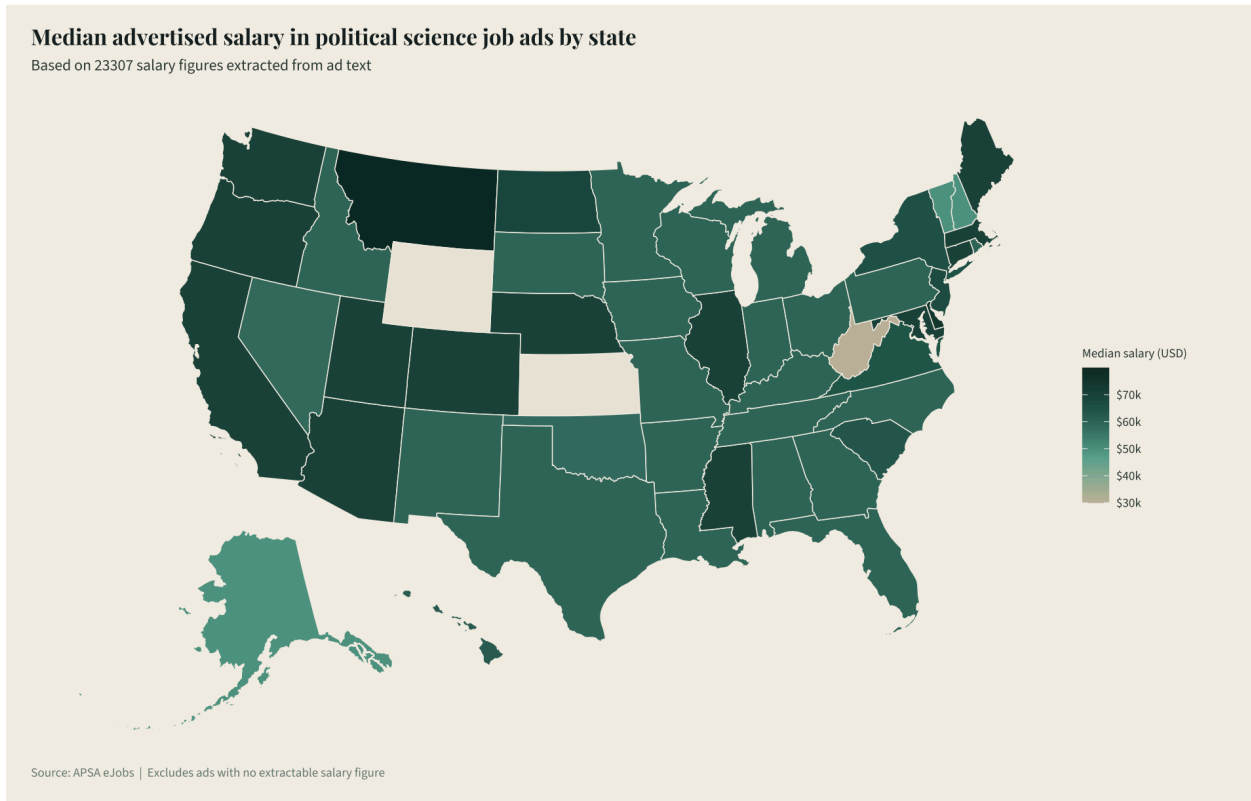
Raw counts across all ads per section



Salary and Pay-Related Mentions: Pre- vs Post-COVID

Raw counts per section split by period





Appendix

A.1 Custom Stopword List

The following terms were removed from the corpus prior to analysis in addition to the standard English stopwords list.

- Foreign language stopwords: French and Spanish function words appeared in addition to English.
- Metadata field labels previously extracted: ejobs, date, deadline, start, salary, rank
- Universal boilerplate: application, applications, applicants, candidates, must, equal, opportunity, affirmative, employer, action, discrimination
- HTML/CSS artifacts: style, margin, font-size, font-family, sans-serif, arial, span
- Institution name artifacts: added iteratively as identified through keyness inspection

A.2 R Code — Preprocessing Pipeline & Corpus Construction

The following code extracts raw text from PDF files, segments individual job ads, assigns section metadata, and extracts structured fields.

```

library(tidyverse);library(pdftools);library(quanteda);library(
quanteda.textstats);library(quanteda.textplots);library(qua
nteda.textmodels);library(topicmodels)

PATH_DATA<-"knowledge_mining/hackathon/job_data"
pdf_files<-list.files(PATH_DATA,pattern="\.pdf$",full.names=
TRUE,ignore.case=TRUE)
safe_pdf_text<-function(path){out<-tryCatch(paste(pdf_text(pa
th),collapse="\n"),error=function(e)
NA_character_);tibble(doc_id=basename(path),text=out,nchar_te
xt=ifelse(is.na(out),NA_integer_,nchar(out)))}
SECTION_PATTERN<-"(?m)^(ADMINISTRATION|AMERICAN GOVERNMENT
AND POLITICS|COMPARATIVE POLITICS|INTERNATIONAL
RELATIONS|METHODOLOGY|NON-ACADEMIC|OTHER|POLITICAL
THEORY|PUBLIC ADMINISTRATION|PUBLIC LAW|PUBLIC POLICY|OPEN)"

extract<-map_dfr(pdf_files,safe_pdf_text)
extract_clean<-extract%>%filter(!is.na(text),nchar_text>0)

extract_ads<-extract_clean%>%
  mutate(ads=map(text,~{
    section_matches<-str_locate_all(.x,SECTION_PATTERN)[[1]]

    section_names<-str_extract_all(.x,SECTION_PATTERN)[[1]]%>%str
_trim()
    chunks<-str_split(.x,"(?<=eJobs
ID:\s{0,5}\\d{1,6}\\s{0,5}\\n)")[[1]]
    chunk_end_positions<-cumsum(nchar(chunks))

    assign_section<-function(pos){if(nrow(section_matches)==0||!a
ny(section_matches[, "start"]<=pos))
return(NA_character_);section_names[max(which(section_matches
[, "start"]<=pos))]}
    tibble(ad_text=chunks,ejobs_id=str_extract(chunks,"eJobs
ID:\s*(\\d+)",group=1),section=map_chr(chunk_end_positions,a
ssign_section))
  })%>%

select(-text)%>%unnest(ads)%>%filter(!is.na(ejobs_id),nchar(a
d_text)>200)%>%
  mutate(
    start_date=str_extract(ad_text,"(?<=Start
Date:)[^\n]+")%>%str_trim(),
    app_deadline=str_extract(ad_text,"(?<=Application
Deadline:)[^\n]+")%>%str_trim(),

```

```

    date_posted=str_extract(ad_text,"(?<=Date
Posted:)[^\n]+") %>%str_trim(),

    salary=str_extract(ad_text,"(?<=Salary:)[^\n]+") %>%str_trim()
,

    institution=str_extract(ad_text,"([^\n]+\n(?:Rank:)",group=1
) %>%str_trim(),

    rank=str_extract(ad_text,"(?<=Rank:)[^\n]+") %>%str_trim(),

    subfields=str_extract(ad_text,"(?<=Subfield\\(s\\):)[^\n]+") %
>%str_trim(),
    date_posted_dt=lubridate::mdy(date_posted),
    year=lubridate::year(date_posted_dt),
    month=lubridate::month(date_posted_dt),

    period=ifelse(date_posted_dt<as.Date("2020-03-01"),"pre_covid
","post_covid")
)

cat("PDFs found:",length(pdf_files),"\\nPDFs
extracted:",nrow(extract_clean),"\\nTotal ads
parsed:",nrow(extract_ads),"\\nAds per PDF - summary:\\n")
extract_ads%>%count(doc_id)%>%pull(n)%>%summary()%>%print()
extract_ads%>%summarise(across(c(institution,rank,subfields,s
ection,start_date,app_deadline,date_posted,salary,date_posted
_dt,year,period),~scales::percent(mean(is.na(.x)),accuracy=.1
)))%>%glimpse()
cat("\\nSection counts (including
NA):\\n");extract_ads%>%count(section,sort=TRUE)%>%mutate(pct=
scales::percent(n/sum(n),accuracy=.1))%>%print(n=Inf)
cat("\\nDate range:\\n
Earliest:",format(min(extract_ads$date_posted_dt,na.rm=TRUE))
, "\\n
Latest:",format(max(extract_ads$date_posted_dt,na.rm=TRUE)), "
\\n\\nAds per year:\\n")
extract_ads%>%count(year,period)%>%print(n=Inf)
set.seed(42);extract_ads%>%slice_sample(n=10)%>%select(doc_id
,ejobs_id,section,institution,rank,date_posted_dt,period)%>%p
rint(width=Inf)
cat("\\nFull text of one
ad:\\n");cat(extract_ads$ad_text[sample(nrow(extract_ads),1)])

inst_sfx<-paste0("(?:University|College|Institute|School|Acad
emy|Institution|Brookings|RAND|Naval|Council|Foundation)")

```

```

eeo_pat<-paste0("((?:[A-Z][\\w&','-]*\\s+){1,7}",inst_sfx,"(?:
[\\w\\s,.-]{0,30})?)\\s+is an? [Ee]qual [Oo]ppportunity")
seeks_pat<-paste0("((?:[A-Z][\\w&','-]*\\s+){1,7}",inst_sfx,"(
?:[\\w\\s,.-]{0,20})?)\\s+(?:seeks|invites|encourages|welcome
s|is seeking|is accepting)")
is_a_pat<-paste0("([A-Z][\\w\\s&','-]{3,55})\\s+is
an?\\s+[\\w\\s]{0,25}(?:[Uu]niversity|[Cc]ollege|[Ii]nstitut|
[Aa]cademy)")
prev_right_pat<-paste0("(?m)\\s{25,}([A-Z][A-Za-z.,'&\\s-]{3,
55}",inst_sfx,"[A-Za-z.,'&\\s-]{0,20})\\s*\\n\\s{10,}Rank:")

recover_body_text<-function(ad_text){
  h<-str_match(ad_text,eeo_pat)[,2];if(!is.na(h))
  return(str_squish(str_remove(h,"^The\\s+")))
  h<-str_match(ad_text,seeks_pat)[,2];if(!is.na(h))
  return(str_squish(str_remove(h,"^The\\s+")))
  h<-str_match(ad_text,is_a_pat)[,2];if(!is.na(h))
  return(str_squish(str_remove(h,"^The\\s+")))
  NA_character_
}

extract_ads<-extract_ads%>%

mutate(institution_clean=if_else(!is.na(institution),str_squi
sh(str_extract(institution,"^[^\\n]+")),NA_character_))%>%

group_by(doc_id)%>%mutate(prev_right_col=str_squish(str_match
(lag(ad_text),prev_right_pat)[,2]))%>%ungroup()%>%

mutate(institution_recovered=case_when(!is.na(institution_cle
an)~NA_character_,!is.na(prev_right_col)~prev_right_col,TRUE~
map_chr(ad_text,recover_body_text)),

institution_clean=coalesce(institution_clean,institution_reco
vered))

extract_ads%>%summarise(total=n(),originally_na=sum(is.na(ins
titution)),recovered_seq=sum(is.na(institution)&!is.na(prev_r
ight_col)),recovered_text=sum(is.na(institution)&is.na(prev_r
ight_col)&!is.na(institution_recovered)),total_recovered=sum(
is.na(institution)&!is.na(institution_clean)),still_na=sum(is
.na(institution_clean)),pct_na_final=scales::percent(mean(is.
na(institution_clean)),accuracy=.1))

```


A.3 R Code —

```

library(usmap)
library(ggrepel)
library(tidyverse)
library(quanteda)
library(quanteda.textstats)
library(quanteda.textplots)
library(ggplot2)
library(showtext)
library(tidytext)

PATH_RDS <- "~/R
Docs/EPPS6323/WordFish/jobads/extract_ads_final.rds"

ads <- readRDS(PATH_RDS)

cat("Loaded", nrow(ads), "job ads\n")
cat("Columns available:", paste(colnames(ads), collapse = ",
"), "\n")

glimpse(ads)

font_add_google("Source Sans 3", "sourcesans")
font_add_google("Playfair Display", "playfair")
showtext_auto()

civic_cat <- c("#2D6A5F", "#D4A843", "#6BAF9E", "#E8C97A",
"#0F3830", "#C8BFA8")

theme_civic <- function(base_size = 11) {
  theme_minimal(base_size = base_size) %+replace%
    theme(
      plot.background = element_rect(fill = "#F5F0E8", color
= NA),
      panel.background = element_rect(fill = "#FFFFFF",
color = NA),
      panel.grid.major = element_line(color = "#C8BFA8",
linewidth = 0.35),
      panel.grid.minor = element_blank(),
      panel.border = element_rect(fill = NA, color =
"#C8BFA8", linewidth = 0.5),
      plot.title = element_text(family = "playfair",
size = base_size * 1.4,
```

```

                                color = "#1C2B2B", face
= "bold", margin = margin(b = 6)),
    plot.subtitle = element_text(size = base_size *
0.95, color = "#3A4A45",
                                margin = margin(b =
12)),
    plot.caption = element_text(family = "sourcesans",
size = base_size * 0.72,
                                color = "#7A8C85",
hjust = 0, margin = margin(t = 8)),
    axis.title = element_text(family = "sourcesans",
size = base_size * 0.78,
                                color = "#7A8C85"),
    axis.text = element_text(family = "sourcesans",
size = base_size * 0.75,
                                color = "#3A4A45"),
    axis.ticks = element_line(color = "#C8BFA8"),
    legend.background = element_rect(fill = "#F5F0E8",
color = NA),
    legend.text = element_text(family = "sourcesans",
size = base_size * 0.75,
                                color = "#3A4A45"),
    strip.background = element_rect(fill = "#EDE8DC", color
= "#C8BFA8"),
    strip.text = element_text(family = "sourcesans",
size = base_size * 0.78,
                                color = "#1C2B2B"),
    plot.margin = margin(16, 16, 16, 16)
  )
}

ads <- ads %>%
  mutate(doc_id = paste0("ad_", row_number()))

job_corpus <- corpus(ads, text_field = "ad_text", docid_field
= "doc_id")

custom_stops <- c(
  # foreign stopwords
  stopwords("fr"), stopwords("es"),

  # academic year patterns
  "2018-2019", "2019-2020", "2020-2021", "2021-2022",

  # boilerplate legal/admin terms

```

```

    "affirmative", "employer", "action", "send",
    "applications",
    "equal", "opportunity", "discrimination", "regardless",

    # institution name artifacts – add as you spot them
    "hampden-sydney", "kapsarc", "kspp", "fudan", "qss",

    # other noise
    "three", "m",
    # metadata field labels
    "ejobs", "date", "deadline", "start", "salary", "rank",

    # universal boilerplate
    "application", "applications", "applicants", "candidates",
    "must",

    # too universal to be informative
    "university", "college", "department",
    # HTML/CSS artifacts
    "style", "margin", "margin-bottom", "margin-top",
    "font-size",
    "font-family", "sans-serif", "arial", "montserrat", "span",
    "0001pt", "10.0pt", "lt", "gt",

    # association acronyms
    "sfpe", "ias", "idinsight", "aarhus", "earlham"
)

job_tokens <- tokens(
  job_corpus,
  remove_punct      = TRUE,
  remove_numbers    = TRUE,
  remove_symbols    = TRUE,
  remove_url        = TRUE
) %>%
  tokens_tolower() %>%
  tokens_remove(stopwords("en")) %>%
  tokens_remove(custom_stops)

job_dfm <- dfm(job_tokens) %>%
  dfm_trim(min_termfreq = 5, min_docfreq = 2)

cat("\nCorpus built:", ndoc(job_corpus), "documents,",
    nfeat(job_dfm), "unique terms\n")

top_words <- topfeatures(job_dfm, n = 30) %>%

```

```

as.data.frame() %>%
rownames_to_column("term") %>%
rename(count = 2) %>%
arrange(desc(count))

ggplot(top_words, aes(x = reorder(term, count), y = count)) +
  geom_col(fill = "#2D6A5F") +
  coord_flip() +
  labs(
    title = "Most common words in political science job
ads",
    subtitle = "All sections, all years",
    x = NULL,
    y = "Total occurrences",
    caption = "Source: APSA eJobs"
  ) +
  theme_civic()

SECTION_TO_EXPLORE <- "METHODOLOGY"

top_by_section <- job_dfm %>%
  dfm_subset(section == SECTION_TO_EXPLORE) %>%
  topfeatures(n = 20) %>%
  as.data.frame() %>%
  rownames_to_column("term") %>%
  rename(count = 2)

ggplot(top_by_section, aes(x = reorder(term, count), y =
count)) +
  geom_col(fill = "#D4A843") +
  coord_flip() +
  labs(
    title = paste("Top words in", SECTION_TO_EXPLORE,
"ads"),
    x = NULL,
    y = "Total occurrences",
    caption = "Source: APSA eJobs"
  ) +
  theme_civic()

job_dfm_dated <- job_dfm %>%
  dfm_subset(!is.na(docvars(job_dfm, "period")))

textstat_keyness(
  job_dfm_dated,

```

```

    target = docvars(job_dfm_dated, "period") == "post_covid"
  ) %>%
  textplot_keyness(
    n = 20,
    color = c("#2D6A5F", "#D4A843")
  ) +
  labs(
    title = "Words distinctive of post-COVID vs pre-COVID
job ads",
    subtitle = "Green = more common post-COVID | Gold =
more common pre-COVID",
    caption = "Source: APSA eJobs"
  ) +
  theme_civic()

TARGET_SECTION <- "NON-ACADEMIC"

textstat_keyness(
  job_dfm,
  target = docvars(job_dfm, "section") == TARGET_SECTION
) %>%
  textplot_keyness(n = 20, color = c("#2D6A5F", "#D4A843")) +
  labs(
    title = paste("Words distinctive of", TARGET_SECTION,
"ads"),
    subtitle = paste(TARGET_SECTION, "vs all other
sections"),
    caption = "Source: APSA eJobs"
  ) +
  theme_civic()

TERMS <- c("remote", "online", "diversity", "equity", "data")

trend_data <- job_dfm %>%
  dfm_group(groups = year) %>%
  dfm_select(pattern = TERMS) %>%
  convert(to = "data.frame") %>%
  rename(year = doc_id) %>%
  filter(!is.na(year)) %>%
  pivot_longer(-year, names_to = "term", values_to = "count")
%>%
  mutate(year = as.integer(year))

ggplot(trend_data, aes(x = year, y = count, color = term)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +

```

```

scale_color_manual(values = civic_cat) +
labs(
  title      = "Term frequency trends in political science
job ads",
  subtitle   = paste("Tracking:", paste(TERMS, collapse = ",
")),
  x          = "Year",
  y          = "Total occurrences",
  color      = NULL,
  caption    = "Source: APSA eJobs"
) +
theme_civic()

ads_per_year <- ads %>%
  filter(!is.na(year)) %>%
  count(year, name = "n_ads")

trend_data_norm <- trend_data %>%
  left_join(ads_per_year, by = "year") %>%
  mutate(per_1000 = (count / n_ads) * 1000)

ggplot(trend_data_norm, aes(x = year, y = per_1000, color =
term)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_color_manual(values = civic_cat) +
  labs(
    title      = "Term frequency trends (normalized)",
    subtitle   = paste("Occurrences per 1,000 ads – tracking:",
paste(TERMS, collapse = ", ")),
    x          = "Year",
    y          = "Occurrences per 1,000 ads",
    color      = NULL,
    caption    = "Source: APSA eJobs"
  ) +
  theme_civic()

ads_per_year_plot <- ads %>%
  filter(!is.na(year)) %>%
  count(year, name = "n_ads") %>%
  mutate(year = as.integer(year))

ggplot(ads_per_year_plot, aes(x = year, y = n_ads)) +
  geom_col(fill = "#2D6A5F") +
  geom_vline(xintercept = 2020, linetype = "dashed", color =
"#0F3830", linewidth = 0.8) +

```

```

    geom_label(aes(x = 2020, y = max(n_ads) * 0.92, label =
"Peak COVID (2020)"),
              hjust      = -0.08,
              family     = "sourcesans",
              size       = 3,
              color      = "#0F3830",
              fill       = "#F5F0E8",
              label.size = 0.3) +
  geom_text(
    aes(label = n_ads),
    vjust = -0.5,
    family = "sourcesans",
    size   = 3.2,
    color  = "#3A4A45"
  ) +
  scale_x_continuous(breaks = unique(ads_per_year_plot$year))
+
  labs(
    title      = "Number of Political Science Job Ads Posted
Per Year",
    subtitle   = "Total listings across all sections",
    x          = "Year",
    y          = "Number of ads",
    caption    = "Source: APSA eJobs"
  ) +
  theme_civic() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

section_year_counts <- ads %>%
  filter(!is.na(year), !is.na(section)) %>%
  count(year, section, name = "n_ads") %>%
  mutate(year = as.integer(year))

ggplot(section_year_counts, aes(x = year, y = n_ads, color =
section, group = section)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1.8) +
  facet_wrap(~ section, scales = "free_y", ncol = 3) +
  scale_color_manual(values =
colorRampPalette(civic_cat)(n_distinct(section_year_counts$se
ction))) +
  scale_x_continuous(breaks =
unique(section_year_counts$year)) +
  labs(
    title      = "Job ad Postings by Section Over Time",

```

```

    subtitle = "Y-axis is free-scaled per section to
highlight relative trends",
    x        = "Year",
    y        = "Number of ads",
    caption  = "Source: APSA eJobs"
) +
theme_civic() +
theme(
  legend.position = "none",
  axis.text.x     = element_text(angle = 45, hjust = 1),
  strip.text      = element_text(size = 8)
)

ggplot(section_year_counts, aes(x = year, y = n_ads, fill =
section)) +
  geom_area(position = "stack", alpha = 0.85, color =
"#F5F0E8", linewidth = 0.3) +
  scale_fill_manual(values =
colorRampPalette(civic_cat)(n_distinct(section_year_counts$se
ction))) +
  scale_x_continuous(breaks =
unique(section_year_counts$year)) +
  labs(
    title      = "Composition of Job Ad Postings by Section
Over Time",
    subtitle   = "Stacked area – shows both total volume and
share per section",
    x          = "Year",
    y          = "Number of ads",
    fill       = NULL,
    caption    = "Source: APSA eJobs"
) +
theme_civic() +
theme(
  axis.text.x     = element_text(angle = 45, hjust = 1),
  legend.position = "bottom",
  legend.key.size = unit(0.6, "lines")
) +
guides(fill = guide_legend(nrow = 3))

N_YEARS    <- 2

N_TERMS    <- 10

```



```

get_section_keyness <- function(section_name, dfm_obj,
n_years, n_terms) {

  sub_dfm <- dfm_obj %>%
    dfm_subset(section == section_name & !is.na(year))

  if (ndoc(sub_dfm) < 10) return(NULL) # skip tiny sections

  years_present <- sort(unique(as.integer(docvars(sub_dfm,
"year"))))

  if (length(years_present) < (n_years * 2)) return(NULL) #
not enough years

  early_years <- head(years_present, n_years)
  late_years <- tail(years_present, n_years)

  sub_dfm_filtered <- sub_dfm %>%
    dfm_subset(as.integer(year) %in% c(early_years,
late_years))

  if (ndoc(sub_dfm_filtered) < 4) return(NULL)

  is_late <- as.integer(docvars(sub_dfm_filtered, "year"))
%in% late_years

  ks <- tryCatch(
    textstat_keyness(sub_dfm_filtered, target = is_late,
measure = "chi2"),
    error = function(e) NULL
  )

  if (is.null(ks) || nrow(ks) < 2) return(NULL)
  emerging <- ks %>% filter(chi2 > 0) %>% slice_max(chi2, n
= n_terms)
  declining <- ks %>% filter(chi2 < 0) %>% slice_min(chi2, n
= n_terms)

  bind_rows(
    emerging %>% mutate(direction = "Emerging"),
    declining %>% mutate(direction = "Declining")
  ) %>%
    mutate(
      section = section_name,
      early_window = paste(early_years, collapse = "-"),

```

```

    late_window = paste(late_years, collapse = "-")
  )
}

all_sections <- unique(na.omit(docvars(job_dfm, "section")))

keyness_all <- map_dfr(all_sections, ~ get_section_keyness(
  section_name = .x,
  dfm_obj      = job_dfm,
  n_years      = N_YEARS,
  n_terms      = N_TERMS
))

keyness_plot_data <- keyness_all %>%
  filter(!section %in% c("OPEN", "NON-ACADEMIC")) %>%
  mutate(
    chi2_signed = if_else(direction == "Declining",
      -abs(chi2), abs(chi2)),
    direction   = factor(direction, levels = c("Emerging",
      "Declining"))
  ) %>%
  group_by(section, direction) %>%
  slice_max(abs(chi2_signed), n = 4) %>%
  ungroup() %>%
  mutate(term = reorder_within(feature, chi2_signed,
    section))

ggplot(keyness_plot_data, aes(x = chi2_signed, y = term, fill
= direction)) +
  geom_col(width = 0.75) +
  geom_vline(xintercept = 0, color = "#C8BFA8", linewidth =
0.5) +
  scale_y_reordered() +
  scale_fill_manual(
    values = c("Emerging" = "#2D6A5F", "Declining" =
"#D4A843"),
    name   = NULL
  ) +
  facet_wrap(~ section, scales = "free_y", ncol = 2) +
  labs(
    title      = "Emerging and Declining Keywords by Section",
    subtitle   = paste0(
      "Earliest ", N_YEARS, " vs latest ", N_YEARS,
      " years within each section |  $\chi^2$  keyness, target =
recent"
    ),
  ),

```

```

    x      = "← Declining | Emerging →",
    y      = NULL,
    caption = "Source: APSA eJobs"
  ) +
  theme_civic(base_size = 10) +
  theme(
    legend.position = "top",
    legend.key.size = unit(0.6, "lines"),
    strip.text      = element_text(size = 8, face = "bold"),
    axis.text.y     = element_text(size = 7),
    axis.text.x     = element_text(size = 7),
    panel.spacing   = unit(1.2, "lines")
  )

SALARY_TERMS <- c("salary", "pay", "compensation", "stipend",
" wage")

salary_by_section <- job_dfm %>%
  dfm_select(pattern = SALARY_TERMS) %>%
  convert(to = "data.frame") %>%
  bind_cols(section = docvars(job_dfm, "section")) %>%
  filter(!is.na(section), !section %in% c("OPEN",
"NON-ACADEMIC")) %>%
  pivot_longer(-c(doc_id, section), names_to = "term",
values_to = "count") %>%
  group_by(section, term) %>%
  summarise(total = sum(count), .groups = "drop") %>%
  group_by(section) %>%
  mutate(section_total = sum(total)) %>%
  ungroup()

salary_by_section <- salary_by_section %>%
  left_join(ads %>% count(section, name = "n_ads"), by =
"section") %>%
  mutate(total = (total / n_ads) * 1000)

ggplot(salary_by_section, aes(x = reorder(section,
section_total), y = total, fill = term)) +
  geom_col(width = 0.7) +
  coord_flip() +
  scale_fill_manual(values = civic_cat, name = "Term") +
  labs(
    title      = "Salary and Pay-Related Term Mentions by
Section",
    subtitle   = "Raw counts across all ads per section",
    x          = NULL,

```

```

    y      = "Total mentions",
    caption = "Source: APSA eJobs"
  ) +
  theme_civic() +
  theme(
    legend.position = "top",
    legend.key.size = unit(0.6, "lines")
  )

salary_by_section_period <- job_dfm %>%
  dfm_select(pattern = SALARY_TERMS) %>%
  convert(to = "data.frame") %>%
  bind_cols(
    section = docvars(job_dfm, "section"),
    period  = docvars(job_dfm, "period")
  ) %>%
  filter(
    !is.na(section), !is.na(period),
    !section %in% c("OPEN", "NON-ACADEMIC")
  ) %>%
  pivot_longer(-c(doc_id, section, period), names_to =
"term", values_to = "count") %>%
  group_by(section, period, term) %>%
  summarise(total = sum(count), .groups = "drop") %>%
  mutate(period = factor(period, levels = c("pre_covid",
"post_covid"),
                                labels = c("Pre-COVID",
"Post-COVID")))

ggplot(salary_by_section_period, aes(x = period, y = total,
fill = term)) +
  geom_col(width = 0.65, position = "stack") +
  facet_wrap(~ section, scales = "free_y", ncol = 3) +
  scale_fill_manual(values = civic_cat, name = "Term") +
  labs(
    title      = "Salary and Pay-Related Mentions: Pre- vs
Post-COVID",
    subtitle  = "Raw counts per section split by period",
    x         = NULL,
    y         = "Total mentions",
    caption   = "Source: APSA eJobs"
  ) +
  theme_civic(base_size = 10) +
  theme(
    legend.position = "top",
    legend.key.size = unit(0.6, "lines"),

```

```

    strip.text      = element_text(size = 8, face = "bold"),
    panel.spacing   = unit(1.2, "lines")
  )

remote_counts <- job_dfm %>%
  dfm_subset(!is.na(docvars(job_dfm, "period"))) %>%
  dfm_select(pattern = "remote") %>%
  convert(to = "data.frame") %>%
  bind_cols(period = docvars(
    job_dfm %>% dfm_subset(!is.na(docvars(job_dfm,
"period"))),
    "period"
  )) %>%
  group_by(period) %>%
  summarise(
    remote_mentions = sum(remote),
    total_words      = n(),
    .groups          = "drop"
  ) %>%
  mutate(not_remote = total_words - remote_mentions)

# Build the contingency table
contingency_table <- remote_counts %>%
  select(period, remote_mentions, not_remote) %>%
  column_to_rownames("period") %>%
  as.matrix()

# Run the chi-squared test
chi_result <- chisq.test(contingency_table)

# Print readable results
cat("=== Chi-squared test: 'remote' pre- vs post-COVID
===\n\n")
cat("Observed counts:\n")
print(contingency_table)
cat("\nExpected counts (if no difference):\n")
print(round(chi_result$expected, 1))
cat("\nChi-squared statistic:", round(chi_result$statistic,
3), "\n")
cat("Degrees of freedom: ", chi_result$parameter, "\n")
cat("P-value:           ", format.pval(chi_result$p.value,
digits = 4), "\n")
cat("\nConclusion: The difference is",
  if_else(chi_result$p.value < 0.05,
    "STATISTICALLY SIGNIFICANT (p < 0.05)",

```

```

      "not statistically significant (p >= 0.05)"),
"\n")

# — Visualization: "remote" pre- vs post-COVID


---



remote_plot_data <- remote_counts %>%
  mutate(
    period = factor(period,
                     levels = c("pre_covid",
                                "post_covid"),
                     labels = c("Pre-COVID",
                                "Post-COVID")),
    expected = chi_result$expected[, "remote_mentions"]
  ) %>%
  pivot_longer(
    cols = c(remote_mentions, expected),
    names_to = "type",
    values_to = "count"
  ) %>%
  mutate(type = factor(type,
                       levels = c("remote_mentions",
                                   "expected"),
                       labels = c("Observed", "Expected (if
no difference)")))

ggplot(remote_plot_data, aes(x = period, y = count, fill =
type)) +
  geom_col(position = position_dodge(width = 0.6), width =
0.5, alpha = 0.9) +
  geom_text(
    aes(label = round(count, 1)),
    position = position_dodge(width = 0.6),
    vjust = -0.5,
    family = "sourcesans",
    size = 3.2,
    color = "#3A4A45"
  ) +
  annotate(
    "text",
    x = 1.5,
    y = max(remote_plot_data$count) * 1.12,
    label = paste0(" $\chi^2 = ", round(chi_result$statistic, 2),
                    " | p ", ifelse(chi_result$p.value <
0.001, "< 0.001",$ 
```

```

paste0("= ",
round(chi_result$p.value, 3))),
  family = "sourcesans",
  size = 3.2,
  color = "#0F3830"
) +
scale_fill_manual(
  values = c("Observed" = "#2D6A5F",
             "Expected (if no difference)" = "#D4A843"),
  name = NULL
) +
scale_y_continuous(expand = expansion(mult = c(0, 0.18))) +
labs(
  title = "Observed vs Expected Mentions of 'remote'",
  subtitle = "If COVID had no effect, observed and expected
bars would be equal",
  x = NULL,
  y = "Number of mentions",
  caption = "Source: APSA eJobs"
) +
theme_civic() +
theme(
  legend.position = "top",
  legend.key.size = unit(0.6, "lines")
)

salary_extracted <- ads %>%
mutate(
  # Extract all salary-like numbers from the text
  salary_raw = str_extract_all(
    ad_text,
    "\\$[0-9]{2,3}(?:,[0-9]{3})?(?:\\. [0-9]+)?[kK]?|\\b[0-9]{2,3}
,[0-9]{3}\\b"
  )
) %>%
unnest(salary_raw) %>%
mutate(
  # Standardise to numeric
  salary_clean = salary_raw %>%
    str_remove_all("\\$|,|USD") %>%
    str_trim() %>%
    # Convert 80k → 80000
    { ifelse(str_detect(., "[kK]$"),
             as.numeric(str_remove(., "[kK]$")) * 1000,
             as.numeric(.)) }

```

```

) %>%
# Plausible academic salary range: $30k-$300k
filter(salary_clean >= 30000, salary_clean <= 300000)

# — Step 2: Aggregate median salary by state


---



salary_by_state <- salary_extracted %>%
  filter(!is.na(state), state != "") %>%
  group_by(state) %>%
  summarise(
    median_salary = median(salary_clean, na.rm = TRUE),
    mean_salary   = mean(salary_clean,   na.rm = TRUE),
    n_ads         = n_distinct(doc_id),
    .groups       = "drop"
  ) %>%
  # usmap expects full state names or two-letter
  abbreviations
  # if your state column is full names, use as-is
  # if abbreviations, rename to match usmap expectations
  rename(state_col = state)

cat("Sample state values:\n")
print(head(unique(salary_by_state$state_col), 20))

state_map_data <- salary_by_state %>%
  rename(state = state_col)

plot_usmap(
  data      = state_map_data,
  values    = "median_salary",
  color     = "#F5F0E8",
  linewidth = 0.3
) +
  scale_fill_gradientn(
    colors = c("#C8BFA8", "#6BAF9E", "#2D6A5F", "#0F3830"),
    na.value = "#EDE8DC",
    name     = "Median salary (USD)",
    labels   = scales::dollar_format(scale = 1e-3, suffix =
"k")
  ) +
  labs(
    title     = "Median advertised salary in political science
job ads by state",
    subtitle  = paste0("Based on ", nrow(salary_extracted),

```



```

      " salary figures extracted from ad
text"),
  caption = "Source: APSA eJobs | Excludes ads with no
extractable salary figure"
) +
  theme(
    plot.background = element_rect(fill = "#F5F0E8", color =
NA),
    plot.title       = element_text(family = "playfair", size
= 14,
                                     color = "#1C2B2B", face =
"bold",
                                     margin = margin(b = 6)),
    plot.subtitle    = element_text(family = "sourcesans",
size = 10,
                                     color = "#3A4A45", margin
= margin(b = 8)),
    plot.caption      = element_text(family = "sourcesans",
size = 8,
                                     color = "#7A8C85", hjust
= 0),
    legend.background = element_rect(fill = "#F5F0E8", color
= NA),
    legend.text       = element_text(family = "sourcesans",
size = 8,
                                     color = "#3A4A45"),
    legend.title      = element_text(family = "sourcesans",
size = 8,
                                     color = "#3A4A45"),
    legend.position   = "right",
    plot.margin       = margin(16, 16, 16, 16)
  )

top_salary_states <- state_map_data %>%
  filter(!is.na(median_salary), n_ads >= 5) %>% # min 5 ads
for reliability
  slice_max(median_salary, n = 15)

ggplot(top_salary_states, aes(x = reorder(state,
median_salary),
                             y = median_salary)) +
  geom_col(fill = "#2D6A5F", width = 0.7) +
  geom_text(
    aes(label = scales::dollar(median_salary, scale = 1e-3,
suffix = "k", accuracy = 1)),
    hjust = -0.1,

```

```

    family    = "sourcesans",
    size      = 3,
    color     = "#3A4A45"
  ) +
  geom_text(
    aes(label = paste0("n=", n_ads), y = 31000),
    hjust     = 0,
    family    = "sourcesans",
    size      = 2.6,
    color     = "#7A8C85"
  ) +
  coord_flip() +
  scale_y_continuous(
    labels = scales::dollar_format(scale = 1e-3, suffix =
"k"),
    expand = expansion(mult = c(0.05, 0.15))
  ) +
  labs(
    title      = "Top 15 states by median advertised salary",
    subtitle   = "Minimum 5 ads per state | n = number of
salary figures extracted",
    x          = NULL,
    y          = "Median advertised salary",
    caption    = "Source: APSA eJobs"
  ) +
  theme_civic()

```

A.4 R Code — Analysis

The following code produces keyness comparisons between target and reference groups using chi-squared statistics.

```
# textstat_keyness and textplot_keyness
```

```
# dfm_group, dfm_select, and ggplot trend lines
```

A.7 AI-Assisted Code Development: Sample Prompts

The following prompts are representative of queries submitted to Claude (Anthropic, claude-sonnet-4-6) during the development of the data processing and analysis pipeline. Prompts have been lightly edited for clarity. All outputs were reviewed, tested, and modified by the research team prior to use.

```
# I have 113 monthly PDF files of political science job listings from APSA eJobs. Each PDF contains multiple job ads separated by a trailing eJobs ID field. What is the most reliable approach to extract individual ads as the unit of analysis using R and quanteda?
```

```
# My corpus_segment() call is returning zero segments. Here is the raw text structure of one PDF page – the eJobs ID appears after each listing, not before it. How should I restructure the split logic to capture each ad with its own ID as a trailing delimiter?
```

```
# Section headers such as ADMINISTRATION, COMPARATIVE POLITICS, and METHODOLOGY appear once per section in the PDF and apply to all ads beneath them. Because they are not present in the individual ad text after segmentation, how can I assign the correct section label to each ad based on its position relative to the header?
```

```
# My keyness plot for NON-ACADEMIC ads vs all others is dominated by HTML and CSS artifacts – terms like font-size, sans-serif, margin-bottom, and span. These appear to come from ads copy-pasted from formatted emails into the PDFs during extraction. What is the best point in the pipeline to address this, and what should my custom stopword list include?
```